

Ritesh Sharma

Test ID: 450015228506336 | 8950803039 | ritesh.sharma232@lpu.in

Test Date: December 11, 2025

Computer Science

53 / 100



Logical Ability

69 / 100



Computer Programming

57 / 100



Quantitative Ability (Advanced)

57 / 100



English Comprehension

56 / 100



Automata Fix

0 / 100



Automata Pro

4 / 100



Personality

✓ Completed

Computer Science

53 / 100

OS and Computer Architecture

58 / 100

DBMS

64 / 100

Computer Networks

27 / 100

Logical Ability

69 / 100

Inductive Reasoning

71 / 100

Deductive Reasoning

70 / 100

Abductive Reasoning

67 / 100

Computer Programming

57 / 100

Basic Programming

64 / 100

Data Structures

54 / 100

OOP and Complexity Theory

52 / 100

Quantitative Ability (Advanced)

57 / 100

Basic Mathematics

60 / 100

Advanced Mathematics

51 / 100

Applied Mathematics

61 / 100

English Comprehension

56 / 100

CEFR: **B2**

Grammar

59 / 100

Vocabulary

55 / 100

Comprehension

54 / 100

Automata Fix

0 / 100

Code Reuse

0 / 100

Logical Error

0 / 100

Syntactical Error

0 / 100

Automata Pro

4 / 100

Programming Ability

0 / 100

Programming Practices

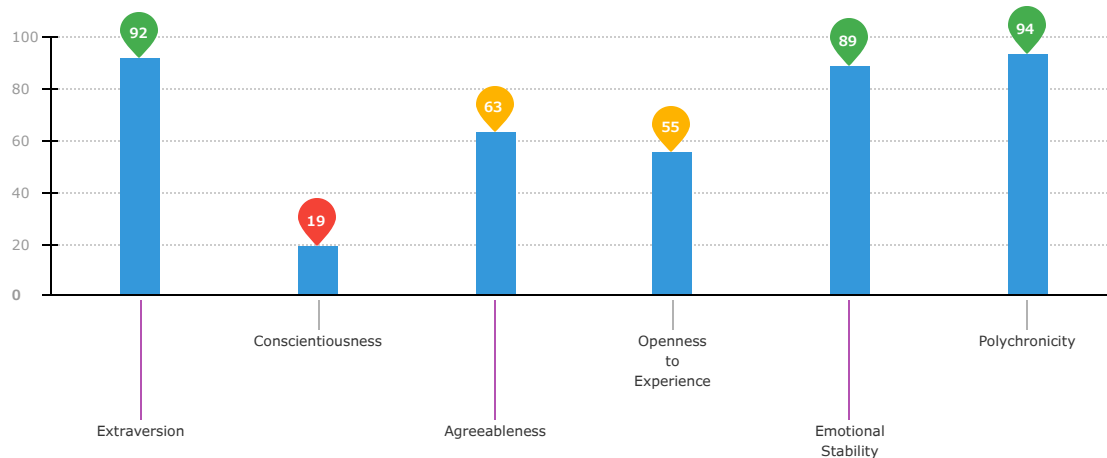
0 / 100

Functional Correctness

10 / 100

Personality

Completed



Competencies

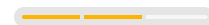
People Interaction



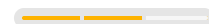
Self-Drive



Trainability



Repetitive Job Suitability



Work attributes

1 | Introduction

About the Report

This report provides a detailed analysis of the candidate's performance on different assessments. The tests for this job role were decided based on job analysis, O*Net taxonomy mapping and/or criterion validity studies. The candidate's responses to these tests help construct a profile that reflects her/his likely performance level and achievement potential in the job role

This report has the following sections:

The **Summary** section provides an overall snapshot of the candidate's performance. It includes a graphical representation of the test scores and the subsection scores.

The **Insights** section provides detailed feedback on the candidate's performance in each of the tests. The descriptive feedback includes the competency definitions, the topics covered in the test, and a note on the level of the candidate's performance.

The **Response** section captures the response provided by the candidate. This section includes only those tests that require a subjective input from the candidate and are scored based on artificial intelligence and machine learning.

The **Learning Resources** section provides online and offline resources to improve the candidate's knowledge, abilities, and skills in the different areas on which s/he was evaluated.

Score Interpretation

All the test scores are on a scale of 0-100. All the tests except personality and behavioural evaluation provide absolute scores. The personality and behavioural tests provide a norm-referenced score and hence, are percentile scores. Throughout the report, the colour codes used are as follows:

● $70 \leq \text{Score} < 100$

● $30 \leq \text{Score} < 70$

● $0 \leq \text{Score} < 30$

2 | Insights

English Comprehension

56 / 100

CEFR: **B2**

This test aims to measure your vocabulary, grammar and reading comprehension skills.

You are able to construct short sentences and understand simple text. The ability to read and comprehend is important for most jobs. However, it is of utmost importance for jobs that involve research, content development, editing, teaching, etc.

Logical Ability

69 / 100



Inductive Reasoning

71 / 100

This competency aims to measure the your ability to synthesize information and derive conclusions.

It is commendable that you have excellent inductive reasoning skills. You are able to make specific observations to generalize situations and also formulate new generic rules from variable data.



Deductive Reasoning

70 / 100

This competency aims to measure the your ability to synthesize information and derive conclusions.

It is commendable that you have excellent inductive reasoning skills. You are able to make specific observations to generalize situations and also formulate new generic rules from variable data.



Abductive Reasoning

67 / 100

Quantitative Ability (Advanced)

57 / 100

This test aims to measure your ability to solve problems on basic arithmetic operations, probability, permutations and combinations, and other advanced concepts.

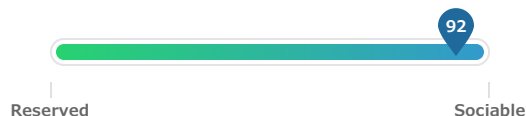
You are able to solve word problems on basic concepts of percentages, ratio, proportion, interest, time and work. Having a strong hold on these concepts can help you understand the concept of work efficiency and how interest is accrued on bank savings. It can also guide you in time management, work planning, and resource allocation in complex projects.

Personality

Competencies



Extraversion



Extraversion refers to a person's inclination to prefer social interaction over spending time alone. Individuals with high levels of extraversion are perceived to be outgoing, warm and socially confident.

- You are outgoing and seek out opportunities to meet new people.
- You tend to enjoy social gatherings and feels comfortable amongst strangers and friends equally.
- You display high energy levels and like to indulge in thrilling and exciting activities.
- You may tend to be assertive about your opinions and prefer action over contemplation.
- You take initiative and are more inclined to take charge than to wait for others to lead the way.
- Your personality is well suited for jobs demanding frequent interaction with people.



Conscientiousness



Conscientiousness is the tendency to be organized, hard working and responsible in one's approach to your work. Individuals with high levels of this personality trait are more likely to be ambitious and tend to be goal-oriented and focused.

- You have an easy going attitude towards work.
- You tend to act spontaneously and may prefer to get started on a project or at work without laying out an organized plan.
- You tend to make quick decisions rather than spending time on reviewing the facts at hand.
- You have a flexible and spontaneous approach to work and is less likely to adhere to strict guidelines and policies.
- You sometimes feel insecure about your capabilities and could benefit from emotional support.
- You are likely to be content with your level of achievement and may not strive to achieve more.
- Your personality is more suited for jobs which demand fast results and are not as concerned with attention to detail.



Agreeableness



Agreeableness refers to an individual's tendency to be cooperative with others and it defines your approach to interpersonal relationships. People with high levels of this personality trait tend to be more considerate of people around them and are more likely to work effectively in a team.

- You are flexible regarding your opinions and be willing to accommodate the needs of others.
- You are generally considerate of the needs of others yet may, at times, overlook social norms to achieve personal success.
- You are selective about the people you choose to trust.
- You are caring and you empathise a friend in distress.
- You give credit to others but also tends to be open with your friends about personal achievements.
- You are more inclined to strike a compromise in tough situations and may be suitable for jobs that demand managing expectations among different stakeholders.



Openness to Experience

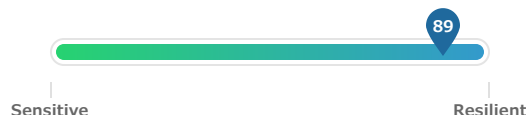


Openness to experience refers to a person's inclination to explore beyond conventional boundaries in different aspects of life. Individuals with high levels of this personality trait tend to be more curious, creative and innovative in nature.

- You may try new things but would prefer not to venture too far beyond your comfort zone.
- You tend to be open to accepting abstract ideas after weighing them against existing solutions.
- You appreciate the arts to a certain extent but may lack the curiosity to explore them in depth.
- You may express your feelings only to people you are comfortable with.
- Your personality is more suited for jobs involving a mix of logical and creative thinking.



Emotional Stability



Emotional stability refers to the ability to withstand stress, handle adversity, and remain calm and composed when working through challenging situations. People with high levels of this personality trait tend to be more in control of their emotions and are likely to perform consistently despite difficult or unfavourable conditions.

- You are calm and composed in nature.
- You tend to maintain composure during high pressure situations.
- You are very confident and comfortable being yourself.
- You find it easy to resist temptations and practice moderation.
- You are likely to remain emotionally stable in jobs with high stress levels.



Polychronicity



Polychronicity refers to a person's inclination to multitask. It is the extent to which the person prefers to engage in more than one task at a time and believes that such an approach is highly productive. While this trait describes the personality disposition of a person to multitask, it does not gauge their ability to do so successfully.

- You pursue multiple tasks simultaneously, switching between them when needed.
- You prefer working to achieve some progress on multiple tasks simultaneously than completing one task before moving on to the next task.
- You tend to believe that multitasking is an efficient way of doing things and prefers an action packed work life with multiple projects.

3 | Response

Automata Pro



4 / 100

[Code Replay](#)

Question 1 (Language: C++20)

You are given a list of numbers. Write an algorithm to remove all the duplicate numbers of the list so that the list contains only distinct numbers in the same order as they appear in the input list.

Scores

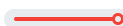
Programming Practices



0 / 100

Programming practices score cannot be generated. This is because source code has syntax/runtime errors and is unparseable or the source code does not meet the minimum code-length specifications.

Functional Correctness



20 / 100

The source code does not pass any basic test cases. It is either due to incorrect logic or runtime errors. Some advanced or edge cases may randomly pass.

Final Code Submitted

Compilation Status: Pass

```
1 // Sample code to read input and write output:
2
3 /*
4 #include <iostream>
5
6 using namespace std;
7
8 int main()
9 {
10     char name[20];
11     cin >> name;          // Read input from STDIN
12     cout << "Hello " << name;    // Write output to STDOUT
13     return 0;
14 }
15 */
16
17 // Warning: Printing unwanted or ill-formatted data to output will cause the test cases to fail
18
19 #include <iostream>
20
21 using namespace std;
22
23 int main()
24 {
25
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code: $O(N)$

*N represents number of elements in the input list.

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilities and Errors

There are no errors in the candidate's code.

```
26 return 0;
27 }
```

Test Case Execution

Passed TC: 0%

Total score

0/20

0%

Basic(0/10)

0%

Advance(0/7)

0%

Edge(0/3)

Compilation Statistics

0

Total attempts

0

Successful

0

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:00:48

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

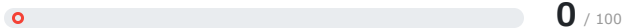
Question 2 (Language: C++20)

Seventh grade students are introduced to the concept of lines in basic geometry class. The class teacher applies this concept in real life. If the teacher considers their own house as one point and the student's house as another point, the teacher can draw a line between these two points. Similarly, if the houses of all students are considered as different points, the teacher can draw multiple lines with their own house as the common point in each line. By taking their own house as the reference, the teacher marks the coordinates of their N students' houses.

Write an algorithm that takes the coordinates of the teacher's house (x_0, y_0) and the students' houses (x_i, y_i) and outputs the number of unique lines that can be drawn with the teacher's house as the common point in each line.

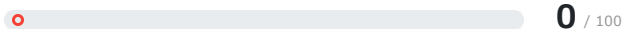
Scores

Programming Ability



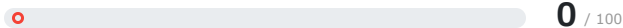
NA

Functional Correctness



NA

Programming Practices



Programming practices score cannot be generated. This is because source code has syntax/runtime errors and is unparseable or the source code does not meet the minimum code-length specifications.

Final Code Submitted

Compilation Status: Pass

```
1 // Sample code to read input and write output:
2
3 /*
4 #include <iostream>
5
6 using namespace std;
7
8 int main()
9 {
10  char name[20];
11  cin >> name;          // Read input from STDIN
12  cout << "Hello " << name;    // Write output to STDOUT
13  return 0;
14 }
15 */
16
17 // Warning: Printing unwanted or ill-formatted data to output will c
18   ause the test cases to fail
19
20 #include <iostream>
21
22 using namespace std;
23
24 int main()
25 {
26  // Write your code here
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code: $O(N)$

*N represents number of friends of Tim

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

```
26 return 0;
27 }
```

Test Case Execution

Passed TC: 0%

Total score

0/11

0%

Basic(0/3)

0%

Advance(0/7)

0%

Edge(0/1)

Compilation Statistics

0

Total attempts

0

Successful

0

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:00:02

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

0%

i Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

i Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Automata Fix



0 / 100

[Code Replay](#)

Question 1 (Language: C++)

The function/method ***allExponent*** returns a real number representing the result of exponentiation of base raised to power exponent for all input values. The function/method ***allExponent*** accepts two arguments - *baseValue*, an integer

representing the base and *exponentValue*, an integer representing the exponent.

The incomplete code in the function/method ***allExponent*** works only for positive values of the exponent. You must complete the code and make it work for negative values of exponent as well.

Another function/method ***positiveExponent*** uses an efficient way for exponentiation but accepts only positive *exponent* values. You are supposed to use this function/method to complete the code in ***allExponent*** function/method.

Helper Description

The following function is used to represent a *positiveExponent* and is already implemented in the default code (Do not write this definition again in your code):

```
int positiveExponent(int baseValue, int exponentValue)
{
    /*It calculates the Exponent for the positive value of exponentValue

    This can be called as -

    int res = (float)positiveExponent(baseValue, exponentValue);*/
}
```

Scores

Final Code Submitted	Compilation Status: Pass	Code Analysis
<pre> 1 // You can print the values to stdout for debugging 2 using namespace std; 3 float allExponent(int baseValue, int exponentValue) 4 { 5 float res = 1; 6 if(exponentValue >=0) 7 { 8 res = (float)positiveExponent(baseValue, exponentValue); 9 } 10 else 11 { 12 // write your code here for negative exponentInput 13 } 14 return res; 15 } 16 17</pre>		Average-case Time Complexity Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases. Best case code: *N represents
		Errors/Warnings There are no errors in the candidate's code.
		Structural Vulnerabilites and Errors There are no errors in the candidate's code.

Test Case Execution

Passed TC: **66.67%**

Total score

4/6

100%

Basic(2/2)

33%

Advance(1/3)

100%

Edge(1/1)

Compilation Statistics

0

Total attempts

0

Successful

0

Compilation errors

1

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:00:30

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

66.7%

i

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

i

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 2 (Language: Java)

The function/method **productMatrix** accepts three arguments - *rows*, an integer representing the rows of the matrix; *columns*, an integer representing the columns of the matrix and *matrix*, a two-dimensional array of integers, respectively.

The function/method **productMatrix** return an integer representing the product of the odd elements whose *i*th and *j*th index are the same. Otherwise, it returns 0.

The function/method **productMatrix** compiles successfully but fails to return the desired result for some test cases. Your task is to debug the code so that it passes all the test cases.

Scores

The candidate did not make any changes in the code.

Compilation Statistics

0

Total attempts

0

Successful

0

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:00:00

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 3 (Language: Java)

You are given predefined structure **Time** containing *hour*, *minute*, and *second* as members. A collection of functions/methods for performing some common operations on times is also available. You must make use of these functions/methods to calculate and return the difference.

The function/method ***difference_in_times*** accepts two arguments - *time1*, and *time2*, representing two times and is supposed to return an integer representing the difference in the number of seconds.

You must complete the code so that it passes all the test cases.

.

Helper Description

The following class is used to represent a Time and is already implemented in the default code (Do not write this definition again in your code):

```
public class Time
{
    public int hour;

    public int minute;

    public int second;

    public Time(int h, int m, int s)
    {
        hour = h;

        minute = m;

        second = s;
    }

    public int compareTo(Object anotherTime)
    {
        /*Return 1, if time1 is greater than time2.

        Return -1 if time1 is less than time2

        or, Return 0, if time1 is equal to time2

        This can be called as -

        * If time1 and time2 are two Time then -

        * time1.compareTo(time2) */
    }

    public void addSecond()
    {
```



```

/* Add one second in the time;

This can be called as -

* If time1 is Time then -

* time1.addSecond() */

}

```

Scores

The candidate did not make any changes in the code.

Compilation Statistics



Total attempts



Successful



Compilation errors



Sample failed



Timed out



Runtime errors

Response time:

00:00:00

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 4 (Language: Java)

The function/method ***replaceValues*** is modifying the input list in such a way - if the length of input list is odd, then all the elements of the input list are supposed to be replaced by 1s and in case it is even, the elements should be replaced by 0s.

For example: given the input list [0 1 2], the function will modify the input list like [1 1 1]

The function/method ***replaceValues*** accepts two arguments - *size*, an integer representing the size of the given input list and *inputList*, a list of integers representing the input list.

The function/method ***replaceValues*** compiles successfully but fails to get the desired result for some test cases due to incorrect implementation of the function. Your task is to fix the code so that it passes all the test cases.

Scores

The candidate did not make any changes in the code.

Compilation Statistics



Total attempts



Successful



Compilation errors



Sample failed



Timed out



Runtime errors

Response time:

00:00:00

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 5 (Language: Java)

The function/method ***sortArray*** modify the input list by sorting its elements in descending order.

The function/method ***sortArray*** accepts two arguments - *len*, representing the length of the list and *arr*, a list of integers representing the input list, respectively.

The function/method ***sortArray*** compiles successfully but fails to get the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

Scores

The candidate did not make any changes in the code.

Compilation Statistics

0

Total attempts

0

Successful

0

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:00:00

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 6 (Language: Java)

The function/method **maxReplace** prints space separated integers representing the input list after replacing all elements of the input list with the maximum element of the input list.

The function/method **maxReplace** accept two arguments - *size*, an integer representing the size of the input list and *inputList*, a list of integers representing the input list, respectively.

The function/method **maxReplace** compiles unsuccessfully due to compilation error. Your task is to debug the code so that it passes all the test cases.

Scores

The candidate did not make any changes in the code.

Compilation Statistics

0

Total attempts

0

Successful

0

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:00:00

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

0%

i Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

i Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 7 (Language: C++)

The function/method ***calculateMatrixSum*** returns an integer representing the sum of odd elements of the given matrix whose j^{th} and j^{th} index are the same.

The function/method ***calculateMatrixSum*** accepts three arguments - *rows*, an integer representing the number of rows of the given matrix, *columns*, an integer representing the number of columns of the given matrix and *matrix*, representing a two-dimensional array of integers.

The function/method ***calculateMatrixSum*** compiles successfully but fails to return the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted

Compilation Status: Pass

```

1 // You can print the values to stdout for debugging
2 using namespace std;
3 int calculateMatrixSum(int rows, int columns, int **matrix)
4 {
5     int i, j, sum=0;
6     if((rows>0) && (columns>0))
7     {
8         for(i=0;i<rows;i++)
9         {
10             sum =0;
11             for(j=0;j<columns;j++)
12             {
13                 if(i==j)
14                 {
15                     if(matrix[i][j]/2!=0)
16                         sum += matrix[i][j];
17                 }
18             }
19         }
20         return sum;
21     }
22     else
23         return sum;
24 }
25

```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilities and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: 12.5%

Total score

1/8

0%

Basic(0/4)

0%

Advance(0/3)

100%

Edge(1/1)

Compilation Statistics

1

Total attempts

1

Successful

0

Compilation errors

1

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:00:37

Average time taken between two compile attempts:

00:00:37

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

4 | Learning Resources

English Comprehension

[Learn about written english comprehension](#)



[Learn about spoken english comprehension](#)



[Test your comprehension skills](#)



Logical Ability

[Learn about advanced deductive logic](#)



[Watch a video on the art of deduction](#)



[Learn about Sherlock Holmes' puzzles and develop your deductive logic](#)



Quantitative Ability (Advanced)

[Watch a video on the history of algebra and its applications](#)



[Learn about proportions and its practical usage](#)



[Learn about calculating percentages manually](#)



Icon Index



Free Tutorial



Paid Tutorial



Youtube Video



Web Source



Wikipedia



Text Tutorial



Video Tutorial



Google Playstore